

Inverse Kinematics Solver Analysis for 6-Axis Industrial Manipulators

Analytical vs. Numerical Approaches, Hybrid Architectures & Solver Selection Rationale

1. Background

This document presents a structured technical assessment of Inverse Kinematics (IK) solving strategies for 6-axis industrial manipulators, with particular reference to the ABB IRB-1300 series. The analysis covers the mathematical properties, operational trade-offs, and deployment considerations for the two canonical solver families—analytical (closed-form) and numerical (iterative)—before examining the compelling case for hybrid architectures and proposing an extended multi-objective validation pipeline. The document concludes with a targeted case study of two state-of-the-art open-source implementations and a brief rationale for the team's initial adoption of a numerical approach.

2. Analytical (Closed-Form) Solvers

Analytical IK solvers exploit the specific geometric structure of a manipulator to derive explicit, algebraic expressions that directly map a desired end-effector pose to all valid joint configurations without any iterative approximation. For a standard 6-DOF robot satisfying the Pieper criterion—where the final three joint axes intersect at a common wrist centre—kinematic decoupling reduces the fully coupled 6-DOF problem into two tractable 3-DOF sub-problems: inverse position kinematics for joints 1–3 and inverse orientation kinematics for the spherical wrist (joints 4–6). The resulting expressions involve only trigonometric operations and the law of cosines, rendering the solution deterministic and $O(1)$ in time complexity.

2.1 Strengths

Analytical solvers deliver several properties that are particularly valuable in high-throughput industrial deployments:

- **Speed.** The solver evaluates a fixed sequence of closed-form expressions. Tools such as IKFast and EAIK routinely achieve execution times well below 100 ms on modern processors. The benchmarking results: <https://nike.box.com/s/8xh09yfo4e55kft69tgf86jqoyqape47> would paint a picture

otherwise, and that is because the full pipeline includes angle normalization, FK verification, filtering)

- **Global solution completeness.** By exhaustively evaluating all branches of the underlying quadratic and trigonometric expressions, an analytical solver simultaneously returns the complete manifold of up to 8 distinct physical configurations (shoulder-left/right, elbow-up/down, wrist-flip/no-flip) in a single pass. Higher-level planners can then select the optimal branch for collision avoidance, joint-limit proximity, or energy minimisation without any additional solver calls. Currently, there are two concrete strategies already implemented: "closest" (angle-wrapped distance to previous config for trajectory continuity) and "min_norm" (minimum joint displacement from zero).
- **Verifiability and certification.** The source code consists entirely of fixed algebraic and trigonometric expressions with no variable-length loops. Bounding worst-case execution time, proving absence of variable overflow, and achieving full structural code coverage are all straightforward tasks, making analytical solvers highly amenable to formal V&V under ISO safety integrity frameworks. And this solver doesn't rely on good random seed, which would often result in non deterministic solution as Jared and Praneeth pointed out in Experiment 12.

2.2 Limitations

Despite their speed and mathematical elegance, analytical solvers carry significant constraints that limit their utility in calibrated or non-standard deployments:

- **Rigidity under calibration offsets.** Analytical equations are derived under the assumption of ideally intersecting wrist axes. Real-world factory calibration routinely reveals sub-millimetre offsets that violate the Pieper criterion. A solver derived from nominal DH parameters will produce a permanent, uncorrectable TCP error on any physical unit whose calibration file introduces these offsets. Re-deriving the full set of equations to accommodate arbitrary spatial offsets is frequently intractable.
- **Mathematical collapse at singularities.** At wrist, shoulder, or elbow singularities, the analytical equations encounter division-by-zero or indeterminate $\text{atan2}(0, 0)$ forms. We need test data collection for singularity to extensively functional test this. Mitigation requires manually authored finite-state-machine logic to detect proximity to each singularity class and clamp specific joint variables—an approach that produces discontinuous joint commands and demands bespoke engineering effort for every singularity configuration.
- **Architectural exclusivity.** Beyond canonical 6R spherical-wrist architectures, deriving closed-form solutions becomes rapidly intractable. A general 6R manipulator with an offset wrist requires extracting roots of a 16th-degree polynomial; 7-DOF or higher redundant arms have no general closed-form solution at all, necessitating a separate solver strategy for any topology change.

- **Numerical Jacobian.** The EAIKFKSolver computes Jacobians via central finite differences ($\text{eps}=1\text{e-}8$), not analytically. This is a significant limitation, as the EAIK backend lacks the Jacobian derivatives and pays a computational and accuracy penalty for Jacobian computation.
- **Angle representation discontinuities.** EAIK returns raw angles outside URDF limits requiring $\pm 2\pi$ normalization

3. Numerical (Iterative) Solvers

Numerical IK solvers treat the inverse kinematics problem as a non-linear root-finding or optimisation task. Starting from an initial seed configuration, the algorithm iteratively evaluates the forward kinematics of the current estimate, computes the task-space error vector $\mathbf{e} = \mathbf{t} - \mathbf{s}(\mathbf{q})$, and applies a correction derived from the manipulator Jacobian $\mathbf{J}(\mathbf{q})$ until $\|\mathbf{e}\|$ falls below a specified tolerance. Four principal Jacobian-based update rules are in common industrial use—the Jacobian Transpose, the Moore-Penrose Pseudoinverse, Damped Least Squares (DLS/Levenberg-Marquardt), and Selectively Damped Least Squares (SDLS)—alongside broader optimisation frameworks such as Sequential Quadratic Programming (SQP) and BFGS that admit arbitrary inequality constraints within the same algorithmic pass.

3.1 Strengths

Numerical solvers offer a complementary profile of advantages that directly addresses the analytical solver's principal weaknesses:

- **Architectural and calibration agnosticism.** Introducing a 0.5 mm calibration offset or modifying a link-twist parameter requires only an update to the DH parameter array or URDF file; the Jacobian automatically reflects the new geometry on the next iteration. The same library operates without modification on 3-DOF planar arms, 6-DOF offset-wrist cobots, and 7-DOF redundant arms. This was one of the experiments that I suggested while we were closing the 2mm gap problem, where we could treat the entire DH parameters as variables of the numerical optimization problem.
- **Intrinsic singularity resilience.** DLS and SDLS monitor the Jacobian's singular values continuously. When the minimum singular value approaches zero, a damping factor λ is introduced into the update equation $\Delta\theta = (\mathbf{J}^T\mathbf{J} + \lambda^2\mathbf{I})^{-1}\mathbf{J}^T\mathbf{e}$, bounding joint velocities to safe magnitudes and enabling the robot to traverse singular zones without software faults or gearbox overload.
- **Native constraint management.** Optimisation-based variants (SQP, BFGS) directly encode mechanical joint limits, self-collision penalties, and manipulability metrics as hard

or soft constraints within the minimised cost function, guaranteeing mechanically feasible solutions even in tightly constrained workspaces.

- **Runtime adaptability.** By adjusting the cost function and constraint weights at runtime, the same solver can balance conflicting objectives—TCP accuracy, joint limit margins, obstacle clearance—without architectural changes, making it well suited to dynamic collaborative environments.

3.2 Limitations

The flexibility of numerical solvers carries a corresponding set of engineering penalties that must be actively managed in safety-critical deployments:

- **Non-deterministic execution time.** Convergence speed depends heavily on seed quality and gradient topology. Execution time can range from under 1 ms to beyond 20 ms in challenging configurations, introducing latency and jitter that may violate RTOS control-loop constraints. Enforcing timeout thresholds to protect the RTOS directly compromises the solver's success rate in difficult regions of the workspace.
- **Local minima entrapment.** The 6-DOF kinematic optimisation landscape is highly non-convex. A poorly chosen seed may cause the gradient-descent process to converge to a local minimum, causing the solver to report infeasibility even when a valid configuration exists. Numerical solvers also tend to converge to the single solution nearest the seed, making deliberate branch switching (e.g., elbow-up to elbow-down) operationally difficult.
- **Certification overhead.** The presence of variable-length iterative loops makes it formally difficult to bound worst-case execution time or achieve complete code coverage during V&V audits. Demonstrating RTOS compliance typically requires extensive Monte Carlo simulation across the full workspace rather than formal proof.

4. Comparative Summary

The table below consolidates the key dimensions of comparison across both solver families for rapid reference.

Criterion	Analytical (Closed-Form)	Numerical (Iterative)
Execution time	< 10 ms*, O(1), zero jitter	1–20+ ms, non-deterministic
Solution coverage	All branches (≤ 8) in one pass	Single solution per seed

Criterion	Analytical (Closed-Form)	Numerical (Iterative)
Singularity handling	Manual patching required; risk of crash	DLS/SDLS provide intrinsic damping
Calibration offsets	Fatal to solution validity; requires re-derivation	Absorbed by updating DH/URDF parameters
Architectural scope	6R spherical-wrist only without major rework	Any topology, same codebase
Constraint integration	Not natively supported	Hard/soft constraints via SQP/BFGS
RTOS V&V	Straightforward; static code paths	Requires Monte Carlo bounding protocols
Physical validation	Fails on calibrated units (ideal model assumed)	Absorbs calibration data seamlessly
Recommended use	High-speed trajectory execution on nominal model	Calibrated robots, redundant arms, cobots
Self collision checking	Supported via the decoupled collision checker	Supported via the decoupled collision checker

* *EAIK IKSolver.solve()* includes angle normalization ($\pm 2\pi$ shifts), FK verification against the target, joint-limit filtering, and solution selection — all wrapping the raw EAIK call. So, the current EAIK time benchmarking is higher than 10ms

5. Solver Selection Guidelines

The preceding analysis suggests a set of practical decision criteria for choosing between solver families on any given deployment:

A

Use an analytical solver as the primary executor when operating a standard 6R spherical-wrist robot on its nominal (un-calibrated) model, where sub-millisecond latency is mandatory and the full solution manifold is needed by a higher-level planner.

N

Use a numerical solver as the primary executor when factory calibration offsets are loaded, when the robot topology deviates from a standard spherical wrist, when hard joint-limit or collision constraints must be enforced within the IK layer, or when the system must adapt at runtime to changing kinematic parameters.

H

Use a hybrid architecture whenever the system must simultaneously satisfy real-time determinism requirements and physical calibration accuracy—which is the condition that describes the overwhelming majority of production-grade deployments.

The codebase uses a clean factory pattern (`create_solvers()`) and abstract base classes (`BaseFKSolver`, `BaseIKSolver`) that enforce a unified interface across all solver backends. This architectural pattern directly enables the hybrid architectures discussed in the next section.

6. Hybrid Solver Architectures

Recognising that neither solver family is universally dominant, the frontier of industrial robot kinematics has converged on hybrid architectures that operate both approaches in combination. Two principal patterns have emerged in practice.

6.1 Sequential Execution (Analytical Seeding)

In this pattern, the analytical solver runs first on the nominal kinematic model, generating an initial configuration estimate that is already close to the true target. This estimate is then passed as the seed to a bounded numerical refinement loop—typically Newton-Raphson or SDLS—which converges within one to three iterations to accommodate calibration offsets, soft joint-limit penalties, or minor constraint violations. Because the seed is already in the topological neighbourhood of the global minimum, the numerical stage cannot become trapped in a local minimum and its execution time is tightly bounded, typically below 10 ms in total. The pattern therefore simultaneously eliminates the primary failure mode of pure numerical solvers (local minima and jitter) while correcting the primary failure mode of pure analytical solvers (calibration error).

6.2 Concurrent Execution (TRAC-IK Architecture)

The TRAC-IK algorithm exemplifies a parallel hybrid strategy. Two independent solver threads are launched simultaneously for every IK query: one executing an inverse-Jacobian method with random restarts on local-minimum detection, and a second executing an SQP-based optimiser with a sum-of-squares metric that is specifically designed to navigate the error boundaries created by hard joint limits. The system returns the result of whichever thread converges first and discards the other. Because the Jacobian thread is fast when it succeeds and the SQP thread is robust when the Jacobian thread fails, the combined architecture achieves success rates exceeding 99.5% while maintaining average latency comparable to a standalone iterative

solver. For a 6-axis industrial manipulator operating in a dense, obstacle-rich cell, this architecture reliably prevents dropped trajectory frames due to solver failure.

7. Proposed Multi-Objective IK Validation Pipeline

Beyond selecting a solver type, robust industrial deployment requires that each IK query be evaluated against a broader set of motion quality criteria. The current convention of treating IK as a single-pass task—returning a joint configuration that satisfies the geometric pose constraint—leaves critical properties such as manipulability, singularity proximity, and trajectory continuity unaddressed at the solver level. We propose formalising these properties into a unified, multi-pass validation pipeline that wraps any underlying solver (analytical, numerical, or hybrid).

7.1 Pipeline Architecture

The pipeline operates as a sequence of four ordered stages, each acting as a gate through which every candidate configuration must pass before being committed to the servo layer:

Stage 1 — Feasibility Check

The initial solver call returns a candidate configuration q^* . Before proceeding, the pipeline evaluates the forward kinematics of q^* and computes the residual Cartesian error $\|f(q^*) - T_{\text{target}}\|$. If the residual exceeds a configurable tolerance (for milestone 1 we had 1 mm positional, 0.017 rad orientational), the configuration is rejected and a restart is triggered using a perturbed seed. For analytical solvers, all available branches are evaluated in parallel and infeasible branches are pruned. This stage prevents residual errors arising from floating-point arithmetic in near-degenerate configurations from propagating to the servo layer. The codebase also includes a complete SelfCollisionChecker (core/collision_checker.py) that uses Pinocchio + hpp-fcl with STL mesh collision geometry and a calibration step to exclude structural overlaps.

Stage 2 — Singularity Margin Evaluation

The pipeline computes the minimum singular value $\sigma_{\min}$ of $J(q^*)$ at the candidate configuration. Configurations where $\sigma_{\min}$ falls below a tunable threshold ϵ_{sing} (empirically set as a fraction of the robot's characteristic length) are classified as near-singular and handled according to a configurable policy: either rejected in favour of an alternative branch, refined via a DLS step that explicitly penalises proximity to the singularity, or flagged and accepted with a velocity-clamping constraint applied downstream. Maintaining an explicit singularity margin decouples singularity management from the solver's internal logic, making the behaviour

auditable and policy-configurable at the system level rather than buried in solver-specific heuristics.

Stage 3 — Manipulability Optimisation

Yoshikawa's manipulability measure $w(q) = \sqrt{\det(J(q)J^T(q))}$ quantifies the robot's ability to generate arbitrary end-effector velocities from the current configuration. Among the set of valid, non-singular candidate configurations returned by Stage 2, the pipeline selects the configuration that maximises $w(q)$, or, in a gradient-based variant, applies a null-space projection step (for redundant manipulators) to move the joints in the direction of increasing manipulability without disturbing the end-effector pose. This stage ensures the robot is not inadvertently steered toward workspace boundaries or low-manipulability configurations during trajectory planning, which reduces joint torque demands and increases the effective dexterous workspace.

Stage 4 — Continuity and Branch Consistency Enforcement

For trajectory tracking applications, the selected configuration must be geometrically continuous with the previously committed configuration q_{prev} . The pipeline computes the L2-norm joint displacement $\|q^* - q_{\text{prev}}\|$ and the maximum per-joint angular velocity implied by the time step. Configurations that would require any joint to exceed its rated velocity limit, or that correspond to an implicit branch switch (e.g., a sudden flip between elbow-up and elbow-down without an explicit planner directive), are rejected and the next-best candidate from the manipulability-ranked set is evaluated. If no candidate passes the continuity gate, the pipeline signals the trajectory planner to re-plan rather than commit a configuration that risks physical discontinuity or mechanical shock.

7.2 Integration and Performance Characteristics

The four-stage pipeline is designed to wrap any underlying solver transparently. For an analytical solver, which returns the complete branch set simultaneously, all four stages operate on the full manifold concurrently, with the final output being a single branch selected through the staged filtering process. For a numerical solver, the pipeline can trigger controlled restarts with perturbed seeds or branch-biasing terms when a candidate is rejected at any stage. The overhead of the four evaluations—singular value computation, manipulability evaluation, and continuity check—is on the order of microseconds per candidate, negligible compared to the servo-loop period. Crucially, the pipeline produces structured diagnostic telemetry at each stage, creating an auditable record of the selection rationale for every commanded configuration, which substantially simplifies post-hoc safety analysis and V&V documentation.



The proposed pipeline transforms IK from a binary feasibility query into a structured quality-assurance process. This is particularly valuable when deploying calibrated or redundant robots in human-collaborative cells where neither singularity proximity nor trajectory discontinuities can be tolerated at any point during production operation.

8. Case Study — Pinocchio vs. EAIK

To ground the preceding analysis in concrete implementation terms, we examine two mature, actively maintained open-source C++ libraries that represent the current state of the art in their respective solver families.

8.1 Pinocchio (Numerical)

Developed at LAAS-CNRS, Pinocchio is a comprehensive rigid-body dynamics library built around Roy Featherstone's spatial algebra formalism. Rather than targeting IK specifically, it provides a general numerical substrate—the Recursive Newton-Euler Algorithm, the Articulated-Body Algorithm, analytical Jacobian derivatives via CRTP template metaprogramming, and deep Eigen integration—on top of which IK can be implemented as a Closed-Loop Inverse Kinematics (CLIK) routine. The canonical Pinocchio CLIK loop evaluates the SE(3) spatial error using `pinocchio::log6`, computes the spatial Jacobian via `pinocchio::computeFrameJacobian`, and applies a DLS update:

```
JW = J.T @ W
H = JW @ J + (lam**2) * np.eye(nv)
g = JW @ e
dq = np.linalg.solve(H, g)
```

Its template architecture also provides analytical Jacobian derivatives suitable for higher-order optimal control frameworks such as Crocoddyl, positioning it as more than an IK solver—it is a complete dynamics computation engine. The primary cost is non-deterministic iteration count and the computational weight of SVD-based updates at each step.

8.2 EAIK (Analytical)

Developed at the Technical University of Munich, the Efficient Analytical Inverse Kinematics (EAIK) toolbox automates the derivation of closed-form solutions by scanning the kinematic configuration for intersecting and parallel joint axes and mapping the problem onto a sequence of canonical IK-Geo geometric subproblems (e.g., Paden-Kahan subproblem 2 for a two-axis

intersection). This process—previously requiring 15+ minutes of symbolic computation in tools like IKFast—is completed by EAIK in under 40 μ s at runtime. EAIK returns the full solution manifold with robust handling of workspace boundaries (returning the nearest least-squares solution for out-of-reach targets rather than a NaN), and avoids the local-minima entrapment inherent to iterative approaches. Its primary limitation is sensitivity to calibration offsets that break the axis-intersection assumptions underpinning the geometric subproblem classification.

8.3 Rationale for Starting with Pinocchio

Given the trade-offs documented above, the decision to begin implementation using Pinocchio as the primary IK solver reflects a deliberate strategic choice rather than a preference for any particular solver philosophy. Several converging factors motivate this direction.

First, the physical robots in our deployment will be operated using their factory calibration files from the outset. These calibration offsets—however minute—are sufficient to invalidate the geometric assumptions on which any analytical solver for this platform would be derived, producing systematic TCP errors that cannot be corrected without re-deriving the full equation set. Pinocchio absorbs these offsets trivially by updating the URDF model.

Second, Pinocchio's integration with Crocoddyl and trajectory optimisation frameworks positions IK not as an isolated, fixed-pose query but as one component within a broader optimal control pipeline. This is aligned with the team's longer-term objective of deploying model-predictive control for high-speed Cartesian path tracking.

Third (future consideration), the non-deterministic execution time of Pinocchio's CLIK loop is mitigated in the near term by the multi-objective validation pipeline proposed in Section 7, which includes a continuity gate that prevents outlier solve times from committing unsafe joint commands, and by the concurrent TRAC-IK (not implemented yet) architecture as a fallback for regions where the primary solver's convergence rate degrades. In the medium term, the sequential hybrid pattern—using an EAIK-derived seed to initialise Pinocchio's refinement loop—remains available as an upgrade path once the baseline numerical system has been validated on the physical hardware / RobotStudio and its workspace convergence map is fully characterised.